

Week 11 Tutorial

Perlin noise

Perlin noise, named after its creator Ken Perlin, is a technique used to produce natural-appearing textures on computer-generated surfaces. Unlike standard random numbers, which can appear disjointed and harsh, Perlin noise ensures that each value is related to its neighbors, creating a smooth, continuous variation. This characteristic makes it ideal for simulating phenomena such as landscapes, clouds, and waves. It is also very useful for adding smooth but unpredictable movement and animation to our projects.

Its best to have a look at this practically.

Part 1:

Objective:

In this section we will make a simple sketch that compares perlin noise with random values. We will draw rectangles of different heights to visualise sequential random values and sequential perlin noise values.

Challenges:

- We need to store values from the perlin noise algorithm into one array, and random values in another.
- We need to create a function that can be re-used to display either the random values or the perlin noise values at different positions on the screen,
- We want to draw the rectangles in a 200 pixel wide space, and adjust their width to fit, so it doesn't matter how many we draw.
- We want to use a variable height to draw rectangles, so the bottom of the rectangles is on the same y value but they are different heights.

Solutions:

- We will make our arrays limited by a variable we set in setup so we can change the number of items in all the arrays and the limits of all the for loops easily.
- We will make a drawValues function that has input parameters for a text label, x and y positions and an array of values
- We will draw the rectangles next to each other with a width of $200 / \text{theNumberOfRectangles}$ and negative height value so they are drawn from the bottom up.

Implementation:

```
// We will make some arrays to hold different kinds of random numbers.
let randomNumberArray = [];
let perlinNoiseArray = [];

// We will define the length of the array here so we can change it easily.
// We will use this in our for loops.
let valueArrayLength = 50;

// And now a variable to control how close together the perlin noise values are taken from
// the noise function.
// Try to change this and watch what happens to the noise values.
// It is like zooming in and out of the noise function.
let perlinNoiseStep = 0.1;

function setup() {
  createCanvas(1000, 500);

  // We will fill our array with random numbers between 0 and 1.
  for (let i = 0; i < valueArrayLength; i++) {
    randomNumberArray.push(random(0, 1));
  }

  // Perlin noise, through the noise function, creates a smooth gradient of values, kind of
  // like a waveform.
  // Now let's fill our perlin noise array with values close by each other from the noise
  // function.
  for (let i = 0; i < valueArrayLength; i += perlinNoiseStep) {
    perlinNoiseArray.push(noise(i));
  }

  // We will draw some text so we set the font size once here because it will be the same
  // for all text.
  textSize(20);
}

function draw() {
  // Set up our drawing environment
  background(255);
  noStroke();
  fill(0);

  drawValues("Random values", 50, 70, randomNumberArray);
  drawValues("Perlin noise values", 50, 280, perlinNoiseArray);
}

// A function to draw our array values as rectangles where the height is derived from each
// value of the array.
// This function takes a title, an x and y position, and an array of values to draw.
function drawValues(title, x, y, randomOrNoiseArray) {
  // Let's draw a text title
  text(title, x, y);

  // We will make all the rectangles fit into 200 pixels so the width of each rectangle
  // will be calculated as 200 divided by the number of values in the array.
  // Note the negative sign in the height calculation - this is because the y-axis is
  // inverted in p5.js so we need to multiply by -1 to make the rectangles draw upwards
  for (let i = 0; i < valueArrayLength; i++) {
    rect(x + (i * 200) / valueArrayLength, y + 130, 200 / valueArrayLength, randomOrNoiseArray
[i] * -100);
  }
}
```

Part 2:

Objective:

In this section we will add another view, using the array values from our random and perlin noise data to alter the colours of a strip of rectangles.

Challenges:

- We need to use HSL colours instead of RGB
- We need to create a function that can be re-used to display either the random values or the perlin noise values as coloured rectangles at different positions on the screen,
- Because we will be changing the colours a lot in a loop we need to make sure these colour changes are kept inside the loop and don't apply to things that follow.

Solutions:

- We will use `colorMode(HSB)`; in our setup to switch p5.js to use HSB (hue saturation and brightness) instead of RGB values - because we do this in setup any colour we set in our sketch after this will use HSB not RGB.
- We will make a `drawNumbersAsColours` function that has input parameters for a text label, x and y positions and an array of values.
- We will use `push()` and `pop()` to isolate the colour changes we use for drawing our rectangles.
- We will draw the rectangles next to each other with a width of `200 / theNumberOfRectangles` and we will still use the negative height value for consistency.

Implementation:

```
// We will make some arrays to hold different kinds of random numbers.
let randomNumberArray = [];
let perlinNoiseArray = [];

// We will define the length of the array here so we can change it easily.
// We will use this in our for loops.
let valueArrayLength = 50;

// And now a variable to control how close together the perlin noise values are taken from
// the noise function.
// Try to change this and watch what happens to the noise values.
// It is like zooming in and out of the noise function.
let perlinNoiseStep = 0.1;

function setup() {
  createCanvas(1000, 500);

  // Let's set the colour mode to HSB. This means that we use Hue, Saturation and Brightness
  // to define colours, not RGB.
  // Hue is the colour, saturation is how much of the colour is in the mix and brightness
  // is how bright the colour is.
  colorMode(HSB);

  // We will fill our array with random numbers between 0 and 1.
  for (let i = 0; i < valueArrayLength; i++) {
    randomNumberArray.push(random(0, 1));
  }

  // Perlin noise, through the noise function, creates a smooth gradient of values, kind of
  // like a waveform.
```

```

// Now let's fill our perlin noise array with values close by each other from the noise
// function.
for (let i = 0; i < valueArrayLength; i += perlinNoiseStep) {
  perlinNoiseArray.push(noise(i));
}

// We will draw some text so we set the font size once here because it will be the same
// for all text.
textSize(20);
}

function draw() {
  // Set up our drawing environment
  background(0, 0, 100);
  noStroke();
  fill(0);

  drawValues("Random values", 50, 70, randomNumberArray);
  drawValues("Perlin noise values", 50, 280, perlinNoiseArray);

  drawNumbersAsColours("Random numbers as hue", 300, 70, randomNumberArray);
  drawNumbersAsColours("Perlin noise as hue", 300, 280, perlinNoiseArray);
}

// A function to draw our array values as rectangles where the height is derived from each
// value of the array.
// This function takes a title, an x and y position, and an array of values to draw.
function drawValues(title, x, y, randomOrNoiseArray) {
  // Let's draw a text title
  text(title, x, y);

  // We will make all the rectangles fit into 200 pixels so the width of each rectangle
  // will be calculated as 200 divided by the number of values in the array.
  // Note the negative sign in the height calculation - this is because the y-axis is
  // inverted in p5.js so we need to multiply by -1 to make the rectangles draw upwards
  for (let i = 0; i < valueArrayLength; i++) {
    rect(x + (i * 200) / valueArrayLength, y + 130, 200 / valueArrayLength, randomOrNoiseArray
[i] * -100);
  }
}

// A function to draw the array values as rectangles where the fill is derived from each
// value of the array.
// This function takes a title, an x and y position and an array of values to draw.
function drawNumbersAsColours(title, x, y, randomOrNoiseArray) {
  // Let's draw a text title
  text(title, x, y);

  // We use push here to save the current fill style so we can pop it back after we have
  // drawn the rectangles.
  push();
  for (let i = 0; i < valueArrayLength; i++) {
    // We will make an HSB colour where the hue is derived from each value of the array.
    // And the saturation and brightness are both set to 100 (maximum).
    fill(randomOrNoiseArray[i] * 360, 100, 100);
    rect(x + (i * 200) / valueArrayLength, y + 130, 200 / valueArrayLength, -100);
  }

  // Pop will restore the fill style to what it was before the push.
  pop();
}

```

Part 3:

Objective:

In this section we will add one more feature, we will draw a circle where each frame the size is set from a consecutive value in an array. This will make a great visualisation of how useful perlin

noise is for making smooth but random animations.

Challenges:

- We need to create a function that can be re-used to display a circle influenced by either the random values or the perlin noise values at different positions on the screen,
- We want to iterate through our arrays, advancing one index per frame, and when we get to the end of the array we want to go back to the start again.

Solutions:

- We will use the modulo function to iterate through our array - this is simple and neat. p5.js has a built in variable frameCount, it is just a number that goes up one for every frame (usually 60 times per second). When we use frameCount % array length we will get new value each frame that starts at 0, counts up one by one to the array length and then goes to zero again and will do so forever.

Implementation:

```
// We will make some arrays to hold different kinds of random numbers.
let randomNumberArray = [];
let perlinNoiseArray = [];

// We will define the length of the array here so we can change it easily.
// We will use this in our for loops.
let valueArrayLength = 50;

// And now a variable to control how close together the perlin noise values are taken from
// the noise function.
// Try to change this and watch what happens to the noise values.
// It is like zooming in and out of the noise function.
let perlinNoiseStep = 0.1;

function setup() {
  createCanvas(1000, 500);

  // Let's set the colour mode to HSB. This means that we use Hue, Saturation and Brightness
  // to define colours, not RGB.
  // Hue is the colour, saturation is how much of the colour is in the mix and brightness
  // is how bright the colour is.
  colorMode(HSB);

  // We will fill our array with random numbers between 0 and 1.
  for (let i = 0; i < valueArrayLength; i++) {
    randomNumberArray.push(random(0, 1));
  }

  // Perlin noise, through the noise function, creates a smooth gradient of values, kind of
  // like a waveform.
  // Now let's fill our perlin noise array with values close by each other from the noise
  // function.
  for (let i = 0; i < valueArrayLength; i += perlinNoiseStep) {
    perlinNoiseArray.push(noise(i));
  }

  // We will draw some text so we set the font size once here because it will be the same
  // for all text.
  textSize(20);
}

function draw() {
```

```

// Set up our drawing environment
background(0, 0, 100);
noStroke();
fill(0);

drawValues("Random values", 50, 70, randomNumberArray);
drawValues("Perlin noise values", 50, 280, perlinNoiseArray);

drawNumbersAsColours("Random numbers as hue", 300, 70, randomNumberArray);
drawNumbersAsColours("Perlin noise as hue", 300, 280, perlinNoiseArray);

drawNumbersAsCircleSize("Random numbers as circle size", 600, 70, randomNumberArray);
drawNumbersAsCircleSize("Perlin noise as circle size", 600, 280, perlinNoiseArray);
}

// A function to draw our array values as rectangles where the height is derived from each
// value of the array.
// This function takes a title, an x and y position, and an array of values to draw.
function drawValues(title, x, y, randomOrNoiseArray) {
  // Let's draw a text title
  text(title, x, y);

  // We will make all the rectangles fit into 200 pixels so the width of each rectangle
  // will be calculated as 200 divided by the number of values in the array.
  // Note the negative sign in the height calculation - this is because the y-axis is
  // inverted in p5.js so we need to multiply by -1 to make the rectangles draw upwards
  for (let i = 0; i < valueArrayLength; i++) {
    rect(x + (i * 200) / valueArrayLength, y + 130, 200 / valueArrayLength, randomOrNoiseArray
[i] * -100);
  }
}

// A function to draw the array values as rectangles where the fill is derived from each
// value of the array.
// This function takes a title, an x and y position and an array of values to draw.
function drawNumbersAsColours(title, x, y, randomOrNoiseArray) {
  // Let's draw a text title
  text(title, x, y);

  // We use push here to save the current fill style so we can pop it back after we have
  // drawn the rectangles.
  push();
  for (let i = 0; i < valueArrayLength; i++) {
    // We will make an HSB colour where the hue is derived from each value of the array.
    // And the saturation and brightness are both set to 100 (maximum).
    fill(randomOrNoiseArray[i] * 360, 100, 100);
    rect(x + (i * 200) / valueArrayLength, y + 130, 200 / valueArrayLength, -100);
  }

  // Pop will restore the fill style to what it was before the push.
  pop();
}

// A function to draw a circle where the size is derived from the array values moving along
// the array each frame.
// This function takes a title, an x and y position and an array of values to draw.
function drawNumbersAsCircleSize(title, x, y, randomOrNoiseArray) {
  // Let's draw a text title
  text(title, x, y);

  // frameCount is a variable that is built into p5.js that increments each frame.
  // We will use the remainder operator to loop through the array each frame.
  // It will give us the remainder of the division of frameCount by the length of the array.
  // This will give us a value between 0 and the length of the array no matter how many
  // frames have passed.
  ellipse(x + 150, y + 80, 50 + randomOrNoiseArray[frameCount % valueArrayLength] * 50);
}

```

Part 4:

Objective:

In this section we will make a simple sketch that generates shapes that use perlin noise to move around the screen. The shapes will be made with a class using normalised position and size values so that the sketch will scale dynamically to fit the window size.

Challenges:

- We will need a class that will let us generate different types of shapes.
- We will move the shapes in the x and y axis so we need to get 2 different but related noise values to alter the positions of the shapes.
- We will need to use normalised values for all dimensions so that the whole sketch will scale with changes to the window size.
- We will need to check the aspect ratio of the screen to know where and what size to draw the shapes.

Solutions:

- We will create a randomNoisyShape class that manages all aspects of the shape, it will take a string into the constructor of circle or square to decide what shape to draw.
- The position and size of the shape will be random, and limited to small values representing a percentage of the width and height, making sure that they are limited to fit and show all the shapes on screen.
- The draw function of the randomNoisyShape class will first check the smaller dimension between the canvas width and height; this will be used to scale the normalised size and position values to fit the screen and maintain the correct aspect ratio.
- The final draw positions of the shape will be influenced by values taken from a perlin noise function. For each axis a perlin noise value will be added to the position. The noise value will need to be scaled - remember we are using normalised coordinates so a 0 x position draws an image all the way at the left of the screen and a 1 would draw the image starting at the right edge of the screen, where it is not seen as it is off the canvas. Perlin noise outputs values between 0 and 1 and we only want to move the shapes a little bit, so each instance of the class will have a noiseScale value to reduce the movement. The result of the position + noise x noiseScale will still be a value between 0 and 1, so now we will multiply the result by the size of the smallest dimension of the whole screen, either width or height.
- As we want movement between frames, we will need to sample the noise from further along the noise function each frame, so we will increment the noiseLocation value every frame as well.

Implementation:

```
// This class will create and manage a random shape with noise.
class randomNoisyShape {
  // The constructor will take a type of shape as an argument - circle or square
  constructor(type) {
    this.type = type;
    // Randomise the x and y position of the shape. These are limited to 80% of the width and height
    this.x = random(0.8);
    this.y = random(0.8);

    // Randomise the size of the shape between 5% and 20% of the width
    this.size = random(0.05, 0.2);
    // Randomise the colour of the shape
    this.colour = [random(255), random(255), random(255)];
    // Randomise the noise scale between 0.1 and 0.35 - this will decide how much the shape will move.
    // Try changing the range to see how it affects the shape's movement.
    this.noiseScale = random(0.1, 0.35);
    // Randomise the noise offset - this is where along the noise function the noise will start to be taken from.
    this.noiseLocation = random(10);
  }

  display() {
    // Use the instance colour to fill the shape.
    fill(this.colour);
    noStroke();
    // Get the smaller dimension of the canvas.
    let minDimension = min(width, height); // Use the smaller dimension to maintain the aspect ratio.
    // Calculate the size of the shape based on the smaller dimension.
    // Remember the sizes were made in percentages, so this will scale correctly no matter what the width.
    let size = this.size * minDimension;

    // We have 2 noise values, one for the x and one for the y. We will sample the yNoise from slightly further along the noise function by adding 10 to the x noise offset.
    let xNoise = noise(this.noiseLocation);
    let yNoise = noise(this.noiseLocation + 10);

    // The x position is the x value of the shape plus the xNoise value multiplied by the noise scale and the width of the canvas - same idea for the y position.
    let x = (this.x + xNoise * this.noiseScale) * width;
    let y = (this.y + yNoise * this.noiseScale) * height;

    // Draw the shape based on the type
    switch (this.type) {
      case 'circle':
        ellipse(x, y, size, size);
        break;
      case 'square':
        rect(x, y, size, size);
        break;
    }
    // Increment the noise offset each frame so we sample the next part of the noise function.
    this.noiseLocation += 0.01;
  }
}

// Create an array to hold the shapes.
let shapes = [];

function setup() {
  createCanvas(windowWidth, windowHeight);

  // Create 4 shapes and add them to the array.
  shapes.push(new randomNoisyShape('circle'));
  shapes.push(new randomNoisyShape('circle'));
  shapes.push(new randomNoisyShape('square'));
  shapes.push(new randomNoisyShape('square'));
}

```



```
function draw() {
  background(0);

  // Use a for-of loop to display the shapes.
  for (let shape of shapes) {
    shape.display();
  }
}

function windowResized() {
  // Resize the canvas when the window is resized. This will update the width and height variables
  // that are used to calculate the size and position of the shapes.
  resizeCanvas(windowWidth, windowHeight);
}
```

Part 5:

Objective:

In this section we will start with the concepts covered last week and extend them with some of our new knowledge. We are going to revisit the transformations [tutorial](https://canvas.sydney.edu.au/courses/72128/pages/week-9-tutorial-2) (<https://canvas.sydney.edu.au/courses/72128/pages/week-9-tutorial-2>) and make some adjustments. Our objective is to first alter the code from the elliptical animation to use a class to create the drawing. Using a class means we can have multiple animations, each having their own properties for defining the drawing, and using normalised coordinates so we can easily scale and position the output. The outcome from this section will use perlin noise to alter the course of the drawing to make more organic, and less predictable lines. The amount that the perlin noise influences the drawing will be controlled by the mouseX position.

Challenges:

- We will need a class that encapsulates all of the functions of the original sketch, but each instance should have their own position, dimensions, speed and rotational offset
- We will need to make the drawing size and position of a class instance react to changes in the browser window size.
- We will need to increment the sampling position from the noise function each frame.

Solutions:

- We will create a DotAnimator class, with position, size, complexity and speed inputs from the constructor to allow for unique variations of the class.
- The DotAnimator class will have an increment variable that we can update when needed, so as not to rely on the frameCount variable and facilitate different speeds for class instances.
- The DotAnimator class will have an angle complexity variable, the angle will be used to control the rotation for each instance, and the complexity will be used to increment the angle.

Because these are class variables it will allow for different instances of the class to rotate different amounts per frame, changing the complexity of the resulting image.

- The noise will be sampled in the main draw loop, outside of the class definition, this means that the update function of the DotAnimator class needs to accept a noise value. This value is used to offset the final circle that is drawn to create the path.
- The DotAnimator class will have a recalculateBounds function, this allows the x and y position and the width and height to be set externally. This is needed so that when the window is resized, instances of the class can get updated sizes and locations.
- In the window resized function, we will first test to find the smaller dimension, width or height, then the smaller dimension will be used to set both the canvas width and height so it is a square. The DotAnimator class instance will then be set to fill the new canvas size.
- Because we are fading the background slowly, whenever we resize the page we will clear the drawing completely and start again.
- To make this easy to see, the class will also draw a rectangle to show the boundaries of each instance.
- To simplify the alignment and scaling, because our canvas is always square, we will alter the css of the p5.js project to centre the canvas on the page. We will use a flexbox layout and align-items: center and justify-content: center to centre the canvas on the page.

Implementation:

First let's change the style.css file to add our flex and centering functionality.

```
html, body {
  /* Set margin and padding to 0 to remove default browser spacing, ensuring the canvas can use the full window size */
  margin: 0;
  padding: 0;
  /* Use flexbox display to facilitate centre alignment of content both horizontally and vertically */
  display: flex;
  /* Align items vertically to the centre of the container */
  align-items: center;
  /* Centre content horizontally within the body */
  justify-content: center;
  /* Set the colour to black to match the canvas */
  background-color: black;
}

canvas {
  display: block;
}
```

Now let's get to the code:

```
// This is the variable we will use to store our class instance.
let animator;
// This is the noise increment value, it will be used to move along the Perlin noise curve
let noiseIncrement = 0;

function setup() {
```

```

// We want to make a square canvas, so we will check the window width and height and take the
// smaller of the two.
let size = Math.min(windowWidth, windowHeight);

// Resize the canvas to be a square using the value we just got.
createCanvas(size, size);

// Set the angle mode to degrees.
angleMode(DEGREES);

// Create a new DotAnimator object. It will be positioned at 0,0 with a size of the width of
// the canvas,
// a complexity of 0.2 and a speed of 1 (it will update every frame).
animator = new DotAnimator(0, 0, width, 0.2, 1);

// Clear the background to black.
background(0);
}

function draw() {
  // Use the alpha value to make the background fade slowly.
  background(0, 1);

  // The noise increment will increase by 0.05 every frame, moving along the Perlin noise curve.
  // Try changing this value to see how it affects the dot's movement.
  noiseIncrement += 0.05;

  // Update the dot animator with the new noise value. We will control how much the
  // noise affects the dot's position with the mouseX value.
  animator.update(noise(noiseIncrement) * (width / mouseX));
}

function windowResized() {
  // Again we want a square, so if the window is resized we will check the new
  // width and height and take the smaller of the two.
  let size = Math.min(windowWidth, windowHeight);

  // Use the new size to resize the canvas.
  resizeCanvas(size, size);

  // Recalculate the bounds of the dot animator - this sets the width and height of the animator.
  animator.recalculateBounds(0, 0, width);

  // Clear the background to black again.
  background(0);
}

function keyPressed() {
  if (key == "c") {
    background(0);
  }
}

class DotAnimator {
  // Our constructor will take the x and y position of the dot, the size of the drawing area
  // (it's a single value because it's a square),
  // the complexity of the dot's movement and the speed of the dot's movement.
  constructor(xPos, yPos, size, complexity, speed) {
    // Use the inputs to the constructor to set the initial values of the class variables.
    this.xPos = xPos;
    this.yPos = yPos;
    this.complexity = complexity;
    this.speed = speed;
    this.size = size;
    // Set the initial angle of the dot to 0.
    this.angle = 0;
    // Set the increment to 0 - we will use this instead of the frameCount so that
    // each instance of the class can have its own increment value.
    this.increment = 0;
    // Draw distortion will influence the dot's position, we will set it to 0 initially.

```

```

    this.drawDistortion = 0;
}

// We have an update function for our class, this will be called every frame.
update(drawDistortion) {
    /*
        Here we can control the speed by controlling when each instance of the class updates its
        increment value.
        Increment value here replaces frameCount in the last version of this code.
        frameCount is a global variable that increases by 1 every frame, our increment only incr
        eases when we want it to.
        We control the increment increase using the remainder operator, this means that the incr
        ement will only
        increase when the frameCount is divisible by the speed value.
    */
    if (frameCount % this.speed == 0) {
        // If we should update the increment value, we increase the increment value by 1.
        this.increment++;
        // The angle will be used to define how many degrees the dot is rotated by (changing the
        complexity of the drawing).
        // We use class variables to control this now.
        this.angle += this.complexity;
    }

    // We will apply the draw distortion even if we don't update the angle and increment.
    // This will make the dot move slightly even when the angle is not changing.
    this.drawDistortion = drawDistortion;

    // We want to draw a bounding box for the instance of the class.
    // We will draw a rectangle at the x and y position with the size of the class.
    stroke(255);
    noFill();
    rect(this.xPos, this.yPos, this.size, this.size); // Draw the boundary at the computed pos
    ition.

    // These translate functions are the same as before, we use 8 and 4 to make an elliptical
    pattern.
    // The only difference is we are using the sin and cos of the increment value instead of t
    he frameCount.
    let translateX = (sin(this.increment) * this.size) / 8;
    let translateY = (cos(this.increment) * this.size) / 4;

    // Now push the drawing context so we can apply transformations to the dot.
    push();
    // First we translate the drawing context to the centre of the class instance position.
    translate(this.xPos + this.size / 2, this.yPos + this.size / 2);
    // Now we rotate as before, but it's now based on our complexity control.
    rotate(this.angle);
    // And we translate with the calculated values just like before.
    translate(translateX, translateY);

    // Draw the animated dot
    noStroke();
    fill(255);
    // Now we draw the dot at 0,0 and apply the distortion value to slightly alter the x and y
    position.
    circle(0 + this.drawDistortion, 0 + this.drawDistortion, 5);
    pop(); // Reset transformations
}

// This function will be used to recalculate the bounds of the class instance,
// so we can react to changes in the canvas size.
recalculateBounds(xPos, yPos, size) {
    this.xPos = xPos;
    this.yPos = yPos;
    this.size = size;
}
}

```

Part 6. Inspiration for the week

This weeks inspiration is a from an Iranian artist called Ali Phi, his work covers, visual art, audio visual installations and music.

In particular I want to draw attention to the work Enfe'al <https://www.aliphi.com/works/enfeal>  (<https://www.aliphi.com/works/enfeal>) the work uses noise to alter positional patterns of pixels in 3d point clouds to create a beautiful effect.

Ali uses noise patterns along with a lot of other complex and well thought out techniques in many of his works, take some time to see some of the other amazing things he has made here:

<https://www.aliphi.com/#works>  (<https://www.aliphi.com/#works>)

Happy coding!!!

Copyright © The University of Sydney. Unless otherwise indicated, 3rd party material has been reproduced and communicated to you by or on behalf of the University of Sydney in accordance with section 113P of the Copyright Act 1968 (Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice.

Live streamed classes in this unit may be recorded to enable students to review the content. If you have concerns about this, please visit our [student guide \(https://canvas.sydney.edu.au/courses/4901/pages/zoom\)](https://canvas.sydney.edu.au/courses/4901/pages/zoom) and contact the unit coordinator.

[Privacy Statement \(https://www.sydney.edu.au/privacy-statement.html\)](https://www.sydney.edu.au/privacy-statement.html)